CHAPTER 05 - RAGAS로 답변 평가하기

RAGAS란?

RAGAS는 `Retrieval-Augmented Generation Assessment`의 약자로,

RAG 시스템의 품질을 평가하기 위한 프레임워크입니다.

여기서 RAG 시스템이란 보통 다음 두 단계를 거칩니다.

1. 사용자의 질문과 관련된 문서를 검색한다.
2. 검색된 문서를 바탕으로 LLM이 답변을 생성한다.

문제는 답변이 별로일 때 원인이 한 가지가 아니라는 점입니다.

- 검색이 잘못되었을 수 있고
- 검색은 잘 되었지만 답변 생성이 문맥을 잘못 사용했을 수 있고
- 질문 의도에 맞지 않게 답했을 수도 있습니다.

RAGAS는 이런 문제를 단순히 "답이 맞다 / 틀리다"로만 보지 않고,

`retrieval`과 `generation`을 분리해서 평가할 수 있게 도와줍니다.

즉, RAGAS는 다음과 같은 질문에 답하기 위한 도구라고 볼 수 있습니다.

- 검색이 잘 되었는가?
- 정답에 필요한 근거를 빠뜨리지 않았는가?
- 검색된 문맥 중 쓸데없는 정보가 너무 많지는 않은가?
- 생성된 답변이 검색 문맥에 충실한가?
- 생성된 답변이 질문 의도에 적절하게 대응했는가?

왜 필요한가?

일반적인 LLM 평가는 보통 최종 답변만 보고 판단합니다.

하지만 RAG 시스템은 답변 이전에 "검색"이라는 별도 단계가 있기 때문에,

최종 출력만 보면 어디서 문제가 생겼는지 알기 어렵습니다.

예를 들어 답변이 틀렸다고 해도:

- 검색 결과에 애초에 필요한 근거가 없었는지
- 필요한 근거는 있었는데 LLM이 잘못 해석했는지
- 질문 의도와 다른 방향으로 답했는지

를 나눠서 봐야 실제 개선 포인트를 찾을 수 있습니다.

RAGAS는 바로 이 지점을 구조적으로 측정하게 해줍니다.

RAGAS가 평가하는 핵심 대상

RAGAS는 보통 아래 요소들을 함께 놓고 평가합니다.

- `user_input` : 사용자 질문
- `reference` : 기준 정답
- `reference_contexts` : 기준 근거 문맥
- `retrieved_contexts` : 실제 검색 결과
- `response` : 실제 생성 답변

이 값을 바탕으로,
"검색 품질"과 "답변 품질"을 함께 판단합니다.

RAGAS의 핵심 장점

- RAG에 특화된 평가 지표를 제공한다.
- retrieval 문제와 generation 문제를 분리해서 볼 수 있다.
- synthetic testset을 생성해 평가 데이터를 빠르게 만들 수 있다.
- 실험 반복과 모델/프롬프트/검색전략 비교에 적합하다.

RAGAS의 한계

- 일부 메트릭은 LLM judge에 의존하므로 평가 결과가 완전히 절대적이지는 않다.
- synthetic testset은 빠르고 편리하지만 실제 사용자 질문 분포를 완벽히 대변하지는 못한다.
- 높은 점수가 실제 서비스 품질을 100% 보장하지는 않는다.
- 따라서 사람 평가, 운영 로그, 사용자 피드백과 함께 보는 것이 가장 좋다.

한 줄로 정리하면

RAGAS는

"RAG 시스템이 왜 잘되거나 왜 망가졌는지"를

검색과 생성 단계로 나눠서 평가하게 해주는 프레임워크입니다.

주요 평가지표

이 노트북에서는 다음 네 가지 지표를 중심으로 봅니다.

- `context_precision`: 검색된 문맥들 중 실제로 관련 있는 문맥이 얼마나 잘 상위에 포함되었는지 평가합니다.
- `context_recall`: 기준 컨텍스트 대비, 필요한 문맥을 얼마나 빠뜨리지 않고 검색했는지 평가합니다.
- `faithfulness`: 생성된 답변이 검색된 문맥에 얼마나 충실한지 평가합니다. 즉, hallucination 여부를 보는 데 유용합니다.
- `answer_relevancy`: 생성된 답변이 질문에 얼마나 적절하게 대응했는지 평가합니다.

이 네 가지를 함께 보면, "검색이 문제인지", "생성이 문제인지", "둘 다 괜찮은데 질문 대응력이 낮은지"를 더 입체적으로 해석할 수 있습니다.

Context Recall은 "정답을 만들 때 필요한 근거를 검색이 얼마나 빠짐없이 가져왔는가"를 보는 지표입니다.

여기서 헷갈리기 쉬운 부분은 문장에 "LLM이 생성한 답변"처럼 보이지만, 실제 설명의 핵심은 생성 답변 자체보다 ground truth(reference) 기준이라는 점입니다.

즉, 보통은:

질문이 있고
정답 역할의 ground truth/reference가 있고
검색된 context가 있을 때
정답에 들어 있는 주장들 중 몇 개가 검색된 context로 뒷받침되는지를 보는 겁니다.

예를 들어볼게요.

질문:
RAGAS의 context recall은 무엇을 측정하나?

ground truth 정답:
- Context recall은 정답에 필요한 근거가 검색 결과에 얼마나 포함되었는지를 측정한다.
- 값은 0에서 1 사이이다.

검색된 context가 이렇게 나왔다고 해봅시다.
- Context recall은 필요한 정보가 retrieval 결과에 포함되었는지를 본다.
- 이 메트릭은 retrieval completeness를 평가한다.

그러면 ground truth의 주장 2개 중:

첫 번째 주장: 검색 결과로 뒷받침 가능
두 번째 주장: 0~1 사이라는 내용은 검색 결과에 없음

그래서 점수는 0.5예요.

핵심 해석은 이렇습니다.

점수가 높다: 검색이 정답에 필요한 근거를 많이 가져왔다
점수가 낮다: 검색 단계에서 필요한 정보가 빠졌다

그래서 이 지표는 답변 생성 성능보다 retrieval 누락 여부를 보는 데 더 가깝다고 이해하면 됩니다.

또 하나 중요한 점:

faithfulness는 "생성된 답변이 검색 context에 충실한가?"
context recall은 "검색 context가 정답을 만들 만큼 충분한가?"

즉 둘은 방향이 달라요.

faithfulness: 답변 → context 기준으로 검증
context recall: 정답(reference) → context 기준으로 검증

지표	무엇을 보나	비교 기준	점수가 높다는 뜻	낮으면 의심할 것
`context_precision`	검색된 문맥이 얼마나 "정확하게" 관련 있었나	`retrieved_contexts` vs 기준 정보	관련 있는 문맥이 상위에 잘 올라옴	검색 결과에 잡음이 많음
`context_recall`	정답에 필요한 근거를 얼마나 "빠짐없이" 가져왔나	`reference` 또는 `reference_contexts` vs `retrieved_contexts`	정답에 필요한 근거가 검색 결과에 충분히 있음	검색 누락이 있음
`faithfulness`	생성 답변이 검색 문맥에 충실한가	`response` vs `retrieved_contexts`	답변의 주장들이 검색 문맥으로 뒷받침됨	hallucination 가능성
`answer_relevancy`	답변이 질문 의도에 잘 맞는가	`user_input` vs `response`	질문에 적절하게 답함	동문서답, 장황함, 초점 이탈

핵심만 더 줄이면 이렇게 보면 됩니다.

- `context_precision`: 검색 결과에 쓸데없는 게 섞였는가
- `context_recall`: 검색 결과가 필요한 근거를 빠뜨렸는가
- `faithfulness`: 답변이 검색 근거 밖으로 튀었는가
- `answer_relevancy`: 답변이 질문에 제대로 답했는가

자주 나오는 해석 패턴도 같이 보면 좋아요.

- `context_recall` 낮음 + `faithfulness` 높음
  검색이 부족함. 답변은 검색된 범위 안에서는 얌전하게 함.
- `context_recall` 높음 + `faithfulness` 낮음
  근거는 충분히 찾았는데 답변 생성이 헛나감. LLM이 문맥을 잘못 쓰거나 지어냄.
- `context_precision` 낮음 + `context_recall` 높음
  필요한 건 가져왔지만 불필요한 문맥도 많이 섞임. retrieval ranking/필터링 개선 여지.
- `answer_relevancy` 낮음 + 나머지 높음
  검색과 사실성은 괜찮은데 질문 의도를 잘못 읽음. 프롬프트나 응답 스타일 문제일 수 있음.

1. 설치

```
pip install -U ragas langchain langchain-openai langchain-communi
ty faiss-cpu pypdf pandas datasets
```

2. 환경 변수

```
import os

os.environ["OPENAI_API_KEY"] = "YOUR_OPENAI_API_KEY"
```

3. 기본 import

```
import pandas as pd

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitt
er
from langchain_community.vectorstores import FAISS

from ragas import EvaluationDataset, evaluate
from ragas.testset import TestsetGenerator
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from ragas.metrics import (
    LLMContextRecall,
    Faithfulness,
    ResponseRelevancy,
    LLMContextPrecisionWithoutReference,
)
```

4. 모델 준비

```
# 실제 RAG 답변 생성용
qa_llm = ChatOpenAI(
    model="gpt-4o-mini",
```

```
    temperature=0,
)

# 평가용 judge 모델
evaluator_llm_raw = ChatOpenAI(
    model="gpt-4o",
    temperature=0,
)

# 임베딩
embedding_model_raw = OpenAIEmbeddings(
    model="text-embedding-3-large"
)

# Ragas wrapper
evaluator_llm = LangchainLLMWrapper(evaluator_llm_raw)
ragas_embeddings = LangchainEmbeddingsWrapper(embedding_model_ra
w)
```

5. PDF 로드

```
pdf_path = "./data/your_document.pdf"

loader = PyPDFLoader(pdf_path)
docs = loader.load()

len(docs), docs[0].page_content[:300]
```

6. 청킹

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
)

split_docs = text_splitter.split_documents(docs)

len(split_docs), split_docs[0].page_content[:300]
```

7. 벡터스토어 생성

```
vectorstore = FAISS.from_documents(
    documents=split_docs,
    embedding=embedding_model_raw,
)

retriever = vectorstore.as_retriever(
    search_kwargs={"k": 4}
)
```

---

8. RAG 실행 함수

```
def format_docs(documents):
    return "\n\n".join(doc.page_content for doc in documents)


def run_rag(question: str):
    retrieved_docs = retriever.invoke(question)
    retrieved_contexts = [doc.page_content for doc in retrieved_d
ocs]

    context_text = format_docs(retrieved_docs)

    prompt = f"""
You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the quest
ion.
If you don't know the answer, just say that you don't know.

Question:
{question}

Context:
{context_text}

Answer:
""".strip()

    response = qa_llm.invoke(prompt).content
```

```
    return {
        "user_input": question,
        "retrieved_contexts": retrieved_contexts,
        "response": response,
    }
```

9. 샘플 질문으로 미리 점검

```
sample_result = run_rag("이 문서의 핵심 내용을 요약해줘.")
sample_result["response"][:500], len(sample_result["retrieved_con
texts"])
```

10. Synthetic testset 생성

가장 단순한 형태로는 generate_with_langchain_docs()를 쓰면 됩니다.

```
generator_llm = evaluator_llm
generator_embeddings = ragas_embeddings

generator = TestsetGenerator(
    llm=generator_llm,
    embedding_model=generator_embeddings,
)

testset = generator.generate_with_langchain_docs(
    split_docs,
    testset_size=10,
)
```

11. 생성된 testset 확인

```
testset_df = testset.to_pandas()
testset_df.head()
```

보통 이런 컬럼을 기대합니다.

- user_input
- reference
- reference_contexts

- 그 외 synthesizer 관련 컬럼

12. testset 질문들에 대해 실제 RAG 실행

```
evaluation_rows = []

for row in testset_df.to_dict(orient="records"):
    rag_result = run_rag(row["user_input"])

    evaluation_rows.append(
        {
            "user_input": row["user_input"],
            "reference": row.get("reference"),
            "reference_contexts": row.get("reference_contexts",
[]),
            "retrieved_contexts": rag_result["retrieved_context
s"],
            "response": rag_result["response"],
        }
    )

evaluation_rows[:2]
```

13. EvaluationDataset 생성

```
evaluation_dataset = EvaluationDataset.from_list(evaluation_rows)
evaluation_dataset
```

14. RAGAS 평가 실행

여기서는 대표적으로 아래 4개를 봅니다.

- LLMContextPrecisionWithoutReference
- LLMContextRecall
- Faithfulness
- ResponseRelevancy

```
metrics = [
    LLMContextPrecisionWithoutReference(llm=evaluator_llm),
```

```
    LLMContextRecall(llm=evaluator_llm),
    Faithfulness(llm=evaluator_llm),
    ResponseRelevancy(llm=evaluator_llm, embeddings=ragas_embeddi
ngs),
]

result = evaluate(
    dataset=evaluation_dataset,
    metrics=metrics,
    llm=evaluator_llm,
    embeddings=ragas_embeddings,
)

result
```

15. 결과를 DataFrame으로 보기

```
result_df = result.to_pandas()
result_df.head()
```

16. CSV 저장

```
result_df = result.to_pandas()
result_df.head()
```

17. 결과 해석용 요약 코드

```
score_cols = [
    col for col in result_df.columns
    if col in [
        "context_precision",
        "context_recall",
        "faithfulness",
        "answer_relevancy",
    ]
]

result_df[score_cols].mean().sort_values(ascending=False)
```